

ch12_gas_processing

```
import jpye
jneqsim = jpye.JPackage("neqsim")

# Assumed NGL recipe for a separate stripping example; not a computed plant outlet
ngl = jneqsim.thermo.system.SystemSrkJEos(273.15 + (-30.0), 25.0)
ngl.addComponent("methane", 5.0)
ngl.addComponent("ethane", 35.0)
ngl.addComponent("propane", 25.0)
ngl.addComponent("i-butane", 8.0)
ngl.addComponent("n-butane", 12.0)
ngl.addComponent("i-pentane", 5.0)
ngl.addComponent("n-pentane", 5.0)
ngl.addComponent("n-hexane", 3.0)
ngl.addComponent("n-heptane", 2.0)
ngl.setMixingRule("classic")

# NGL feed stream
ngl_feed = jneqsim.process.equipment.stream.Stream("NGL Feed", ngl)
ngl_feed.setFlowRate(2000.0, "kg/hr")
ngl_feed.setTemperature(-30.0, "C")
ngl_feed.setPressure(25.0, "bara")

# One contacting stage plus reboiler: a bounded NGL stabilizer, not a product-spec
deethanizer
deethanizer = jneqsim.process.equipment.distillation.DistillationColumn(
    "NGL Stabilizer", 1, True, False)
deethanizer.addFeedStream(ngl_feed, 1) # cold feed at the top
deethanizer.setSolverType(jneqsim.process.equipment.distillation.
DistillationColumn.SolverType.DIRECT_SUBSTITUTION)
deethanizer.setMaxNumberOfIterations(100, True)
deethanizer.setTemperatureTolerance(1e-7)
deethanizer.setMassBalanceTolerance(1e-6)
deethanizer.setEnthalpyBalanceTolerance(1e-5)
deethanizer.setEnforceEnergyBalanceTolerance(True)
deethanizer.setMeshResidualTolerance(1e-5)
deethanizer.setEnforceMeshResidualTolerance(True)

deethanizer.setTopPressure(25.0)
deethanizer.setBottomPressure(26.0)
deethanizer.getReboiler().setHeatInput(100000)
# Specified heat input is 100 kW; product temperatures are calculated.

# Build and run
process = jneqsim.process.processmodel.ProcessSystem()
process.add(ngl_feed)
process.add(deethanizer)
process.run()

# Results
overhead = deethanizer.getGasOutStream()
bottoms = deethanizer.getReboiler().getLiquidOutStream()

print(deethanizer.getConvergenceDiagnostics())
assert str(deethanizer.getLastSolveStatus()) == "RIGOROUS_CONVERGED"
```

```
assert deethanizer.getLastMeshResidualNorm() < 1e-5
assert deethanizer.getGasOutStream().getFlowRate("kg/hr") > 0.0
assert deethanizer.getReboiler().getLiquidOutStream().getFlowRate("kg/hr") > 0.0

print("=== NGL Stripping Results ===")
print(f"Overhead T: {overhead.getTemperature('C'):.1f} C")
print(f"Overhead P: {overhead.getPressure():.1f} bara")
print(f"Bottoms T: {bottoms.getTemperature('C'):.1f} C")
print(f"Reboiler duty: "
      f"{deethanizer.getReboiler().getDuty()/1e3:.1f} kW")
```

ch32_advanced_topics

```
import jpype
import numpy as np
import json
from pathlib import Path
jneqsim=jpype.JPackage('neqsim')
SteadyStateVariable=jneqsim.process.util.reconciliation.SteadyStateVariable
DataReconciliationEngine=jneqsim.process.util.reconciliation.
DataReconciliationEngine
ReconciliationVariable=jneqsim.process.util.reconciliation.ReconciliationVariable
BatchParameterEstimator=jneqsim.process.calibration.BatchParameterEstimator
values=60.0+np.random.default_rng(42).normal(0,0.02,60)
signal=SteadyStateVariable('Synthetic pressure',60)
signal.setWindowSize(60)
for value in values:signal.addValue(float(value))
expected_r=float(np.sum(np.diff(values)**2)/(2*59*np.var(values,ddof=1)))
assert abs(signal.getRStatistic()-expected_r)<1e-10

measured=np.array([100000.0,70000.0,28000.0,5000.0])
sigma=np.array([2000.0,1500.0,1000.0,500.0])
A=np.array([[1.0,-1.0,-1.0,-1.0]])
V=np.diag(sigma**2)
expected=measured-V@A.T@np.linalg.solve(A@V@A.T,A@measured)
engine=DataReconciliationEngine()
names=['Feed','Gas','Oil','Water']
for name,value,sd in zip(names,measured,sigma):
    engine.addVariable(ReconciliationVariable(name,float(value),float(sd)))
engine.addConstraint(jpype.JArray(jpype.JDouble)(A[0].tolist()),'Mass balance')
reconciliation=engine.reconcile()
actual=np.array([engine.getVariable(n).getReconciledValue() for n in names])
assert reconciliation.isConverged()
assert np.max(np.abs(actual-expected))<1e-6 and abs(float((A@actual).item()))<1e-6

def calibration_fixture(eta,discharge):
    fluid=jneqsim.thermo.system.SystemSrKeos(313.15,60.0)
    fluid.addComponent('methane',.9);fluid.addComponent('ethane',.1)
    fluid.setMixingRule('classic')
    feed=jneqsim.process.equipment.stream.Stream('Feed',fluid)
    feed.setFlowRate(100000.0,'kg/hr')
    comp=jneqsim.process.equipment.compressor.Compressor('Compressor',feed)
    comp.setUsePolytropicCalc(True);comp.setPolytropicEfficiency(eta)
    comp.setOutletPressure(discharge,'bara')
    process=jneqsim.process.processmodel.ProcessSystem()
    process.add(feed);process.add(comp);process.run()
    hout=float(comp.getOutletStream().getFluid().getEnthalpy())
    hin=float(feed.getFluid().getEnthalpy());power=float(comp.getPower())
    assert power>0 and abs(hout-hin-power)/power<1e-5
    assert abs(comp.getOutletStream().getFlowRate('kg/hr')-100000.0)<1e-5
    return process,comp

true_eta=.78;observations=[]
for discharge in [130.0,150.0,170.0]:
    _,comp=calibration_fixture(true_eta,discharge)
    observations.append((discharge,float(comp.getOutletStream().getTemperature(
'K'))))
```

```

process,comp=calibration_fixture(.72,150.0)
estimator=BatchParameterEstimator(process)
estimator.addTunableParameter('Compressor.polytropicEfficiency','- ',.60,.90,.72)
estimator.addMeasuredVariable('Compressor.outletStream.temperature','K',1.0)
HashMap=jpye.JClass('java.util.HashMap')
for discharge,temperature in observations:
    conditions=HashMap();conditions.put('Compressor.outletPressure',jpye.JDouble(
discharge))
    observed=HashMap();observed.put('Compressor.outletStream.temperature',jpye.
JDouble(temperature))
    estimator.addDataPoint(conditions,observed)
estimator.setMaxIterations(40);fit=estimator.solve()
estimate=float(fit.getEstimate(0))
assert fit.isConverged() and .60<=estimate<=.90
assert abs(estimate-true_eta)<.002
replay_errors=[]
for discharge,temperature in observations:
    _,independent=calibration_fixture(estimate,discharge)
    replay_errors.append(abs(independent.getOutletStream().getTemperature('K')-
temperature))
assert max(replay_errors)<.02
contract=dict(R=expected_r,reconciliation_max_error=float(np.max(np.abs(actual-
expected))),
              estimated_efficiency=estimate,maximum_replay_error_K=max(replay_
errors),
              native_fit_converged=bool(fit.isConverged()))
Path('ch32_local_calibration_contract.json').write_text(json.dumps(contract,
indent=2))
print(json.dumps(contract,indent=2))

```

ch34_case_studies

```
"""Reduced NGL pressure/capacity screen with explicit external refrigeration."""
def build_gas_plant(feed_rate_kghr):
    # Fresh fluid and equipment for every point.
    fluid=jnqsim.thermo.system.SystemSrEos(303.15,70.0)
    recipe={'nitrogen':.004,'CO2':.030,'methane':.800,'ethane':.070,
            'propane':.040,'i-butane':.010,'n-butane':.015,
            'i-pentane':.008,'n-pentane':.006,'n-hexane':.008,
            'n-heptane':.005,'n-octane':.004}
    for name,fraction in recipe.items():fluid.addComponent(name,fraction)
    fluid.setMixingRule('classic')
    ps=ProcessSystem()
    def run_add(unit):ps.add(unit);unit.run();return unit
    feed=Stream('Plant Feed',fluid)
    feed.setFlowRate(feed_rate_kghr,'kg/hr');run_add(feed)
    inlet_sep=run_add(Separator('Inlet Separator',feed))
    precooler=Cooler('External Refrigeration',inlet_sep.getGasOutStream())
    precooler.setOutTemperature(243.15);run_add(precooler)
    pre_ko=run_add(Separator('Expander Inlet K0',precooler.getOutletStream()))
    expander=jnqsim.process.equipment.expander.Expander('Turboexpander',pre_ko.
getGasOutStream())
    expander.setIsentropicEfficiency(.85);expander.setOutletPressure(22.0);run_
add(expander)
    cold_sep=run_add(Separator('Cold Separator',expander.getOutletStream()))
    recomp=Compressor('First Residue Stage',cold_sep.getGasOutStream())
    recomp.setUsePolytropicCalc(True);recomp.setPolytropicEfficiency(.78)
    recomp.setOutletPressure(35.0,'bara');run_add(recomp)
    residue_comp=Compressor('Residue Compressor',recomp.getOutletStream())
    residue_comp.setUsePolytropicCalc(True);residue_comp.setPolytropicEfficiency(.
78)
    residue_comp.setOutletPressure(70.0,'bara');run_add(residue_comp)
    ps.run()
    liquids=[inlet_sep.getLiquidOutStream(),pre_ko.getLiquidOutStream(),cold_sep.
getLiquidOutStream()]
    products=liquids+[residue_comp.getOutletStream()]
    for stream in products+[feed]:stream.getFluid().initProperties()
    duties=[float(precooler.getDuty()),float(expander.getPower()),
            float(recomp.getPower()),float(residue_comp.getPower())]
    checks=process_boundary_check(feed,products,duties)
    assert expander.getPower()<0 and recomp.getPower()>0 and residue_comp.
getPower()>0
    assert pre_ko.getGasOutStream().getFluid().getNumberOfPhases()==1
    assert cold_sep.getGasOutStream().getFluid().getNumberOfPhases()==1
    row=dict(feed_kghr=feed_rate_kghr,
            feed_MSm3day=float(feed.getFlowRate('MSm3/day')),
            inlet_gas_MSm3day=float(inlet_sep.getGasOutStream().getFlowRate(
'MSm3/day')),
            expander_MW=-float(expander.getPower())/1e6,
            residue_comp_MW=float(residue_comp.getPower())/1e6,
            first_residue_stage_MW=float(recomp.getPower())/1e6,
            refrigeration_MW=-float(precooler.getDuty())/1e6,
            liquid_kghr=sum(float(s.getFlowRate('kg/hr')) for s in liquids),
            checks=checks)
    return ps,row
```

```

# Explicit mass-throughput basis; no universal MMscfd/kg conversion is assumed.
design_rate=290000.0
plant_design,ngl_base=build_gas_plant(design_rate)
plant_new,ngl_increased=build_gas_plant(1.2*design_rate)
ngl_limits={'inlet_gas_MSm3day':10.5,'expander_MW':4.5,'residue_comp_MW':18.0}
for row in [ngl_base,ngl_increased]:
    row['screen_utilization']={k:row[k]/v for k,v in ngl_limits.items()}
    row['screen_feasible']=all(v<=1 for v in row['screen_utilization'].values())
Path('ch34_ngl_base_increased.json').write_text(json.dumps([ngl_base,ngl_
increased],indent=2))
print(json.dumps([ngl_base,ngl_increased],indent=2))

```

long-token-regression

```
value = "verylongidentifierverylongidentifierverylongidentifierverylongidentifier  
verylongidentifierverylongidentifierverylongidentifierverylongidentifier  
verylongidentifierverylongidentifierverylongidentifierverylongidentifier  
verylongidentifierverylongidentifierverylongidentifierverylongidentifier  
verylongidentifierverylongidentifier"  
if value:  
    result = value
```

java-layout-regression

```
if (active) {  
    final String label = "Figure 24.2: literal code string must remain verbatim";  
    column.setSolverType(neqsim.process.equipment.distillation.DistillationColumn.  
SolverType.DIRECT_SUBSTITUTION);  
    logger.info("Run result {}", label);  
}
```